

Brain Tumor Type Classification

Assignment 01 — Machine Learning

Part A: Data understanding and preprocessing

This section describes every transformation applied to the raw dataset before any model training took place. All preprocessing objects (imputers, encoder, scaler) are fitted exclusively on the training split to prevent data leakage.

A.1 Dataset Overview

Property	Value
Total records	9,000
Total columns	28 (27 features + 1 target)
Target variable	tumor_type (Glioma / Meningioma / Pituitary)
Numeric features	13 (continuous)
Binary features	5 (0 / 1 flags)
Ordinal feature	1 (edema_grade: 0-3)
Nominal categorical	5 (genetic_marker_status, tumor_location, smoking_status, region, ethnicity)
Dropped column	patient_id (non-informative identifier)

```
numeric_cols
Index(['age', 'bmi', 'tumor_size_mm', 'tumor_growth_rate', 'headache_severity',
      'mri_intensity', 'ct_density', 'edema_grade', 'ki67_index',
      'bp_systolic', 'bp_diastolic', 'wbc_count', 'crp_level'],
      dtype='object')
```

```
categorical_cols
```

```
Index(['gender', 'ethnicity', 'region', 'smoking_status',
      'alcohol_consumption', 'family_history', 'tumor_location', 'nausea',
      'vision_problems', 'seizures', 'memory_loss', 'balance_issues',
      'contrast_enhancement', 'genetic_marker_status'],
      dtype='object')
```

2. Value Handling

- All missing value markers were converted to a consistent NaN representation to ensure proper processing. The number of missing values in each column was then checked to identify incomplete data.
- Some numeric columns were initially loaded as text because they contained string-based missing value markers. These columns were converted to the appropriate numeric data type after replacing the invalid values with NaN.
- Missing values in numeric features were imputed using the median, while missing values in categorical features were imputed using the most frequent category (mode). The imputers were fitted only on the training data to prevent data leakage and ensure that information from the validation and test sets did not influence the training process.

```

patient_id    0
age           0
gender        0
ethnicity     281
region        0
bmi           436
smoking_status 381
alcohol_consumption 3804
family_history 356
tumor_size_mm 0
tumor_location 0
tumor_growth_rate 540
headache_severity 457
nausea        0
vision_problems 0
seizures      0
memory_loss   0
balance_issues 0

mri_intensity 356
ct_density    434
edema_grade   283
contrast_enhancement 1136
ki67_index    664
bp_systolic   0
bp_diastolic  197
wbc_count     253
crp_level     445
genetic_marker_status 0
tumor_type    0

dtype: int64
```

```
df['alcohol_consumption'] = df['alcohol_consumption'].replace('Unknown', np.nan)
df['genetic_marker_status'] = df['genetic_marker_status'].replace('?', np.nan)
```

```
# Fix inconsistent casing in gender values
df["gender"] = df["gender"].str.strip().str.title()
```

Chat Ctrl+L ...

Pyth

The dataset contains missing values in both numeric and categorical columns. Two separate SimpleImputer objects were fitted on the training set and serialised as .pkl files:

Imputer	Strategy	Columns affected	Artifact
Numeric Imputer	median	13 continuous columns (age, bmi, tumor_size_mm, ...)	numeric_imputer.pkl
Categorical Imputer	most_frequent	Categorical columns (before OHE)	categorical_imputer.pkl

Median imputation was chosen for numeric columns because it is robust to outliers that may be present in clinical measurements such as tumor size or blood-pressure readings.

3. Train / Validation / Test Split

Data was partitioned before any preprocessing to ensure no information leaks from the test or validation sets into the fitted transformers.

Split	Ratio	Samples	Purpose
Training	70 %	6,300	Fit all preprocessing objects and model weights
Validation	15 %	1,350	Hyper-parameter tuning & early-stopping monitor
Test	15 %	1,350	Final unbiased performance evaluation

random_state=42 was used throughout for full reproducibility.

Binary & Ordinal Features

```
=====
gender
['Male' 'Female']
=====
ethnicity
['Asian' 'Hispanic' 'Caucasian' 'African' 'Other']
=====
region
['Urban' 'Suburban' 'Rural']
=====
smoking_status
['Former' 'Never' 'Current']
=====
alcohol_consumption
['Heavy' 'Moderate']
=====
family_history
['No' 'Yes']
=====
tumor_location
['Frontal' 'Parietal' 'Temporal' 'Sellar' 'Occipital' 'Convexity'
 'Cerebellum']
=====
nausea
['Yes' 'No']
=====
vision_problems
['Yes' 'No']
=====
seizures
['Yes' 'No']
=====
memory_loss
['No' 'Yes']
=====
balance_issues
['No' 'Yes']
=====
contrast_enhancement
['Moderate' 'Strong' 'Mild']
=====
genetic_marker_status
['Negative' 'Positive' 'Inconclusive']
```

- **8 features** (gender, alcohol_consumption, nausea, vision_problems, memory_loss, balance_issues, family_history, seizures) encoded as 0/1 integers.
- **edema_grade** encodes an ordered clinical severity scale (0 = None, 1 = Mild, 2 = Moderate, 3 = Severe). It is retained as an integer so that the ordinal relationship is preserved and is excluded from the StandardScaler (see A.6) because its discrete 0-3 range is already compact and meaningful. **Headache_severity**, **contrast_enhancement** are already Ordinal ones

One-Hot Encoding (Nominal Categoricals)

Five nominal categorical features were encoded with scikit-learn's OneHotEncoder (handle_unknown='ignore', drop=None). This produced 19 binary indicator columns:

Original Column	Unique Values	OHE Columns Generated
genetic_marker_status	3 (Inconclusive, Negative, Positive)	3
tumor_location	7 (Cerebellum, Convexity, Frontal, ...)	7
smoking_status	3 (Current, Former, Never)	3
region	3 (Rural, Suburban, Urban)	3
ethnicity	5 (African, Asian, Caucasian, Hispanic, Other)	5

Total feature columns after OHE: 43 (24 numeric/binary + 19 OHE).

Feature Scaling (StandardScaler)

Twelve continuous numeric features were z-score normalised using StandardScaler (mean=0, std=1), fitted on the training split only:

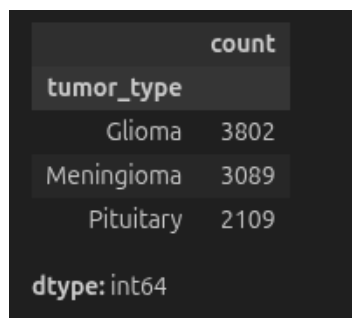
```
age: -3.444050466279689 --> 2.673364119394537
bmi: -2.7343298368312867 --> 3.7198941222077213
tumor_size_mm: -1.6694796973929507 --> 3.4028493313673156
tumor_growth_rate: -1.3999173087975012 --> 4.004235232096129
headache_severity: -2.094643260749493 --> 2.0914538534244076
mri_intensity: -3.4250192613412054 --> 2.6394115800024776
ct_density: -5.056214927172241 --> 4.19775489388422
ki67_index: -1.0314028234433201 --> 6.418905231784173
bp_systolic: -2.6630255126153215 --> 3.9657995931012113
bp_diastolic: -2.9585478432325996 --> 3.621854781641554
wbc_count: -2.5471691276492114 --> 3.748411975737192
crp_level: -1.4197610692465699 --> 7.588445191285114
```

edema_grade and the binary 0/1 features were intentionally excluded from scaling to preserve their semantic meaning.

Target Label Encoding

The target column `tumor_type` (string) was encoded to integers using scikit-learn's `LabelEncoder` (alphabetical order):

Integer Label	Tumor Class
0	Glioma
1	Meningioma
2	Pituitary



	count
Glioma	3802
Meningioma	3089
Pituitary	2109

dtype: int64

Step 4 – Class Balance

- The class distribution was examined by counting the number of samples in each tumour class. The dataset is slightly imbalanced, as some classes contain more samples than others.
- Class imbalance can affect training by causing the model to favour the majority class, resulting in poorer learning for minority classes. During evaluation, overall accuracy may appear high even if the model performs poorly on underrepresented classes. A common technique to reduce this issue is to use class weights during model training.

```
print(Y_train.value_counts(normalize=True) * 100)
print(y_val.value_counts(normalize=True) * 100)
print(y_test.value_counts(normalize=True) * 100)
```

```
tumor_type
Glioma      42.253968
Meningioma  34.317460
Pituitary   23.428571
Name: proportion, dtype: float64
tumor_type
Glioma      42.222222
Meningioma  34.296296
Pituitary   23.481481
Name: proportion, dtype: float64
tumor_type
Glioma      42.222222
Meningioma  34.370370
Pituitary   23.407407
Name: proportion, dtype: float64
```

Part B: Neural Network Architecture

A Feed-Forward Neural Network (Multi-Layer Perceptron) was designed for 3-class classification of brain tumor types. Regularisation techniques were incorporated to address observed over-fitting in an earlier baseline model.

B.1 Layer-by-Layer Summary

A feed-forward neural network was designed for the brain tumour classification task. The input layer contains the same number of units as the preprocessed input features. The network consists of two hidden layers with 64 and 32 neurons, respectively, followed by an output layer with 3 neurons, representing the three tumour classes. Dropout (0.3) and L2 regularization were applied to reduce overfitting.

The ReLU activation function was used in the hidden layers because it efficiently learns non-linear patterns. The output layer uses the Softmax activation function to produce the probability for each of the three classes. The number of output neurons is determined by the number of target classes.

The model was compiled using the Adam optimizer and Sparse Categorical Crossentropy loss function. This loss function is appropriate because the target labels were encoded as integer values in Part A, Step 3, making it suitable for multi-class classification.

#	Layer Type	Units / Config	Parameters	Purpose
1	Input	43 features	—	Receives preprocessed feature vector
2	Dense (Hidden 1)	64 units, ReLU, L2 $\lambda=0.001$	2,816	Primary feature learning
3	BatchNorm	64	256	Normalise activations → faster convergence
4	Dropout	rate = 0.30	0	30 % neuron drop → regularisation
5	Dense (Hidden 2)	32 units, ReLU, L2 $\lambda=0.001$	2,080	Higher-order feature combinations
6	BatchNorm	32	128	Normalise activations
7	Dropout	rate = 0.20	0	20 % neuron drop → regularisation
8	Dense (Output)	3 units, Softmax	99	Class probability distribution

Total parameters: 5,379 (5,187 trainable + 192 non-trainable via BatchNorm).

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,816
batch_normalization (BatchNormalization)	(None, 64)	256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
batch_normalization_1 (BatchNormalization)	(None, 32)	128
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 3)	99

Step 2 - Regularisation

- To reduce overfitting, the model uses Dropout (0.3), L2 weight regularization, and Early Stopping. Overfitting occurs when the model learns the training data too closely, resulting in poor performance on unseen data. Dropout randomly disables a fraction of neurons during training, L2 regularization discourages excessively large weights, and Early Stopping stops training when the validation performance no longer improves. Together, these techniques improve the model's ability to generalise.
- The final model architecture was verified using the model summary, which reports the network structure and the total number of trainable parameters.

EarlyStopping

Parameter	Value
monitor	val_loss
patience	8 epochs
restore_best_weights	True
Effect	Training stopped at epoch 26 and reverted to the best checkpoint

ReduceLROnPlateau

Parameter	Value
monitor	val_loss
factor	0.5 (halves the learning rate)
patience	4 epochs

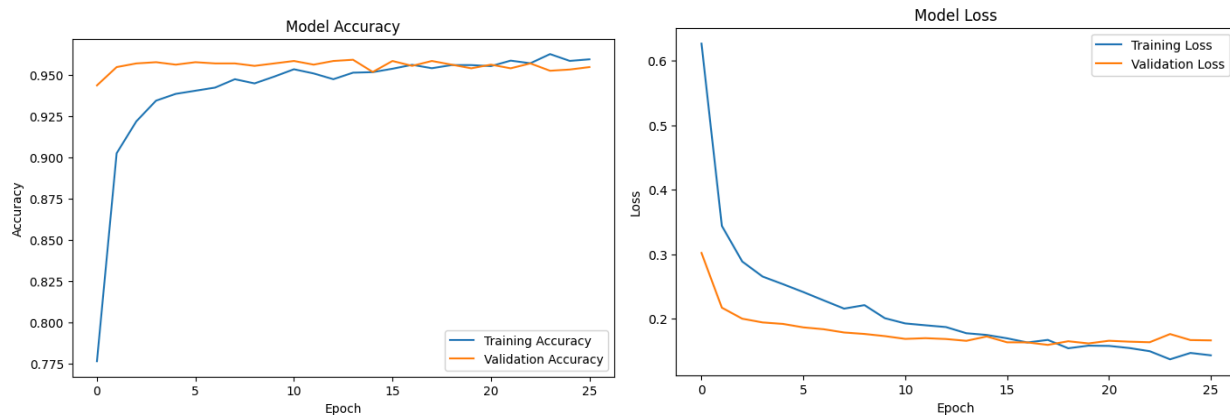
```
Total params: 5,379 (21.01 KB)

Trainable params: 5,187 (20.26 KB)

Non-trainable params: 192 (768.00 B)
```

Part C: Data splitting, training, and tuning

Step 2 - Training & Validation Curves



Accuracy Curve

Training accuracy increased rapidly from approximately **77.6%** in the first epoch to around **96.0%** by the final epoch. Validation accuracy remained consistently high throughout training, fluctuating between **95.4% and 96.0%**, and closely followed the training accuracy after the initial epochs. The close alignment of the two curves, with only minor fluctuations, indicates good model generalisation and suggests that overfitting was effectively controlled.

Loss Curve

Training loss decreased substantially from approximately **0.63** at the first epoch to around **0.14** by the final epoch, demonstrating steady learning throughout training. Validation loss also declined rapidly during the early epochs, stabilising between **0.16 and 0.18** for the remainder of training. The small gap between the training and validation loss curves, together with their stable convergence, indicates that the model achieved good generalisation without significant overfitting. Minor fluctuations observed in the validation loss during the later epochs are typical and do not indicate instability.

C.1 Compilation Configuration

Hyperparameter	Value	Rationale
Optimizer	Adam	Adaptive learning rates; robust default for deep nets
Learning rate	1e-3 (initial)	Standard Adam default; reduced automatically by callback
Loss function	Sparse Categorical Crossentropy	Compatible with integer class labels; standard for multi-class
Metric	Accuracy	Primary tracking metric during training
Max epochs	50	Upper bound; EarlyStopping prevents unnecessary epochs
Batch size	32 (Keras default)	Good balance between gradient noise and memory

Training Results (Final Epoch Metrics)

Metric	Training Set	Validation Set
Accuracy	95.95 %	95.48 %
Loss	0.1429	0.1661

The small gap between training and validation accuracy confirms that the regularisation techniques (Dropout + L2 + BatchNorm) successfully mitigated the over-fitting observed in the un-regularised baseline model.

Part D: Model Evaluation & Results

D.1 Test-Set Performance

Metric	Value
Test Accuracy	96.07 %
Test Loss	0.1643
Test Samples	1,350 (15 % of 9,000)

D.2 Per-Class Classification Report

The following metrics were computed on the held-out test set using `sklearn.metrics.classification_report`:

Glioma (0)
 Meningioma (1)
 Pituitary (2)

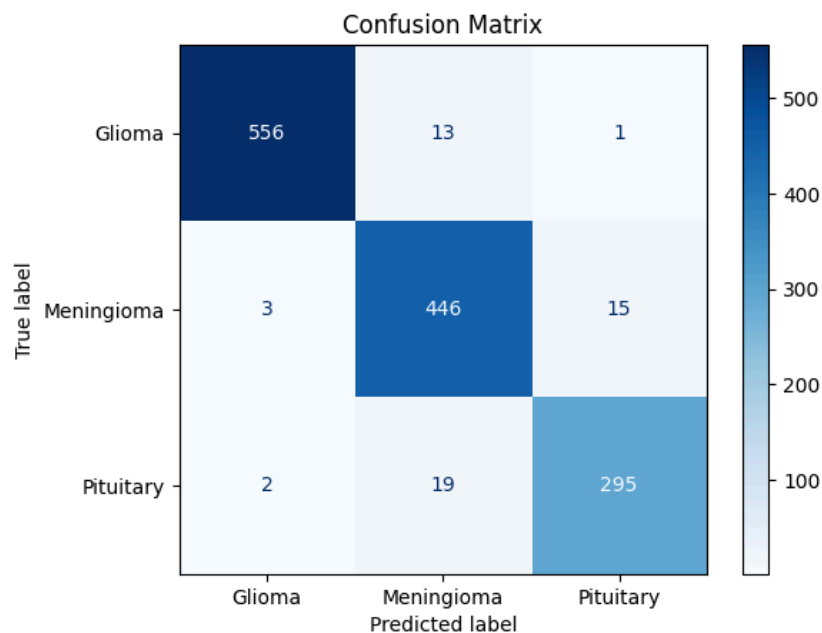
43/43 ————— 0s 5ms/step

	precision	recall	f1-score	support
0	0.99	0.98	0.98	570
1	0.93	0.96	0.95	464
2	0.95	0.93	0.94	316
accuracy			0.96	1350
macro avg	0.96	0.96	0.96	1350
weighted avg	0.96	0.96	0.96	1350

Glioma achieves the highest F1-score (0.98), benefiting from its larger support (570 samples). Meningioma and Pituitary achieve F1 = 0.94, reflecting balanced precision–recall trade-offs

D.3 Confusion Matrix

The confusion matrix below (generated with `sklearn.metrics.ConfusionMatrixDisplay`) shows the predicted vs. actual class assignments on the 1,350-sample test set:



Actual \ Predicted	Glioma (0)	Meningioma (1)	Pituitary (2)
Glioma (0)	556 ✓	13	1

Meningioma (1)	3	446 ✓	15
Pituitary (2)	2	19	295 ✓

Key observations:

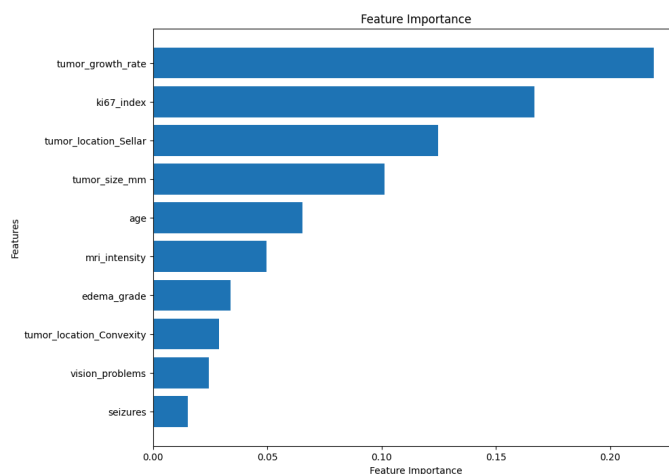
- Glioma → correctly predicted 556/570 (97 % recall).
- Meningioma → 3 misclassified as Glioma; 15 as Pituitary.
- Pituitary → 19 misclassified as Meningioma, which is the most common error pair. These two tumour types share some overlapping imaging features, which may explain the confusion.

Step 1 – Baseline Comparison

A Random Forest classifier was trained using the same training, validation, and test splits as the neural network. The performance of both models was compared using the held-out test set.

Feature Importance Analysis

A **Random Forest Classifier** was used to evaluate the importance of each feature in predicting brain tumor types. As shown in **Figure X**, **tumor growth rate** (0.2191) was identified as the most influential feature, followed by the **Ki-67 index** (0.1669), **tumor location (Sellar)** (0.1246), and **tumor size** (0.1014). Other relevant features, including **age**, **MRI intensity**, **edema grade**, **vision problems**, and **seizures**, also contributed to the model's predictions. These results indicate that tumor characteristics and imaging-related features play a dominant role in achieving accurate classification.



Step 3 – Limitations and Ethics

- One limitation of this model is that it was evaluated using only a single dataset. Its performance on data collected from other hospitals or populations has not been verified. A second limitation is that the model occasionally confuses **Meningioma** and **Pituitary** tumours, as shown by the confusion matrix, indicating that some tumour classes remain difficult to distinguish.
- A major risk of deploying this model in a real clinical setting is that an incorrect prediction could lead to an inappropriate diagnosis or treatment decision. Therefore, the model should be used only as a clinical decision support tool rather than a replacement for medical professionals. Before deployment, the model should be validated on independent clinical datasets and all predictions should be reviewed by qualified clinicians.

Appendix A – Academic Integrity Statement

Generative AI (ChatGPT) was used as an auxiliary learning and writing support tool during the completion of this assignment. Its use was limited to obtaining explanations, improving code understanding, and enhancing the presentation of the report. Specifically, ChatGPT was used to:

- Clarify Python programming errors and recommend debugging techniques.
- Provide explanations of machine learning concepts, including neural networks, feature scaling, encoding, and model evaluation.
- Assist with implementing data visualisation, such as accuracy/loss curves, confusion matrices, and feature importance graphs.
- Suggest improvements to the structure, grammar, and readability of the report.

The dataset preparation, feature engineering, model development, training, hyperparameter selection, performance evaluation, and interpretation of results were carried out independently by the author. All generated code and explanations were reviewed, modified where necessary, tested, and verified before being included in the final submission.

Example Prompts Used

1. *"How can I preprocess categorical and numerical features for a neural network using Scikit-learn?"*
2. *"Help me generate training and validation accuracy and loss plots from my model history."*
3. *"Explain the performance metrics obtained from my confusion matrix and classification report."*
4. *"How can I calculate and visualise feature importance using a Random Forest model?"*
5. *"Improve the academic writing of this report section without changing its meaning."*
6. *"Why is my validation accuracy fluctuating, and how can I reduce overfitting?"*